

Likelihood II

Today's agenda:

- Estimating complex functions

Last time

- Likelihood
- Functions
- Different options for optimizing (optim, mle2, brute force)
- How to identify the parameter for a simple model that maximizes likelihood
 - But we only did models with a single parameter

What do we do when there are lots of parameters?

Let's return to the tadpole predation data

Remember, the experiment involved different treatments:

- Tadpole size variation
- Predation (presence/absence)
- Stocking density

Last time we picked one density level

- What if we want to understand how density impact predation rate?

What do we do when there are lots of parameters?

Load and plot the dataset:

```
data("ReedfrogFuncresp")
```

Plot both:

Number Killed ~ Initial density

Per capita death rate ~ Initial density

Predation model

We'll model predation using a type II functional response

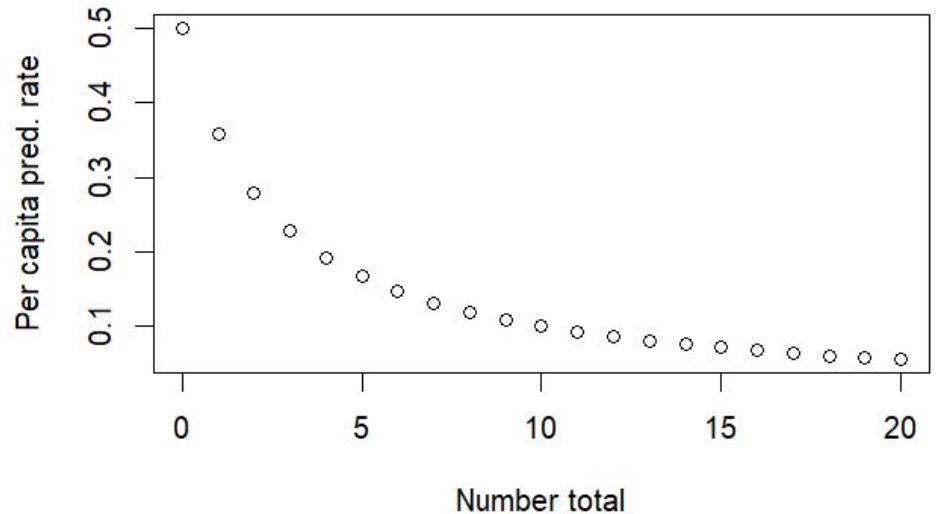
$$p = a / (1 + a * h * N)$$

p = predation rate

a = attack rate

h = handling time

N = number of prey



Predation model

We'll model predation using a type II functional response

$$p = a / (1 + a * h * N)$$

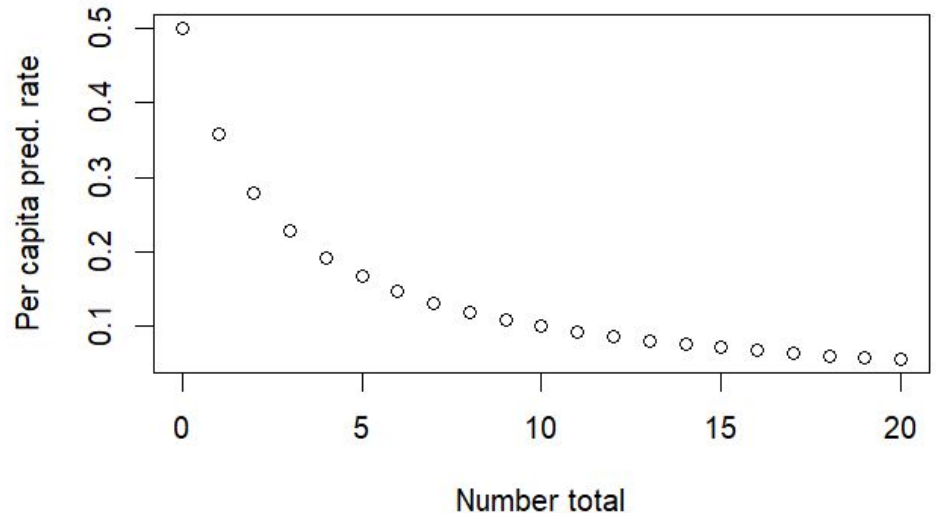
p = predation rate

a = attack rate

h = handling time

N = number of prey

Attack rate is lessened by the amount of time it takes to consume prey. If handling time is zero, $p = a$.



Understanding the Predation model

When you come across a new model, it can be useful to play with it a bit to understand the parameters

- Try plotting $p \sim N$
- Try different values for a and h to understand how they control the behavior
- What does “ a ” do? What does “ h ” do?

```
a <- .5  
h <- .8  
N_vec <- 0:20
```

```
plot(x = N_vec, y = a/(1+a*h*N_vec),  
      ylab = "Per capita pred. rate",  
      xlab = "Number total")
```

$$p = a / (1 + a * h * N)$$

p = predation rate

a = attack rate

h = handling time

N = number of prey

Predation model

We'll model predation using a type II functional response

$$p = a / (1 + a * h * N)$$

We'll use a binomial distribution for p (since prey are either eaten or not)

Translating the predation model to R

Predation:

$$\text{pred_prob} = a / (1 + a * h * N)$$

Likelihood:

$$\text{dbinom}(x = k, \text{size} = N, \text{prob} = \text{pred_prob})$$

We know k and N , are estimating a and h which tells us p

Reminder:

p = predation rate

a = attack rate

h = handling time

N = number of prey

K = number killed

Translating the predation model to R

```
binomNLL2 <- function(params, N, k){  
  a = params[1]      Note that since we're fitting two parameters,  
  h = params[2]      we use a vector for the params  
  pred_prob = a / (1+a*h*N)  
  -sum(dbinom(x = k, size = N, prob = pred_prob, log = TRUE))  
}
```

Optimizing with optim

```
data("ReedfrogFuncresp")  
optim(fn = binomNLL2,  
      par = c(a=.5,h=.2),  
      N = ReedfrogFuncresp$Initial,  
      k = ReedfrogFuncresp$Killed  
      )
```

Optimizing with mle2

Note: mle2 requires you to name parameters of a NLL fx with multiple parms

```
parnames(binomNLL2) <- c("a","h")
```

```
m2 <- mle2(minuslogl = binomNLL2,  
  start = c(a=.5,h=2),  
  data = list(k = ReedfrogFuncresp$Killed,  
    N = ReedfrogFuncresp$Initial))
```

Checking our results

What are the estimated values for a and h ?

Did the two approaches give the same values?

Checking our results

It's helpful to plot your results to see if they make sense

```
plot( y = ReedfrogFuncresp$Killed / ReedfrogFuncresp$Initial,  
      x = ReedfrogFuncresp$Initial,  
      xlab = "Initial Density",  
      ylab = "Per capita predation rate" )
```

Checking our results

Now, add your predicted lines

- Combine the equation with fitted parameters

First, we need a vector of x values to evaluate at

```
x_vec <- min(ReedfrogFuncresp$Initial):max(ReedfrogFuncresp)
```

Checking our results

Now, plug our fitted values into the equation:

$$p = a / (1 + a * h * N)$$

```
y_vec_optim <- opt2$par["a"] /  
                (1 + opt2$par["a"] * opt2$par["h"] * x_vec)
```


Checking our results

Add this fitted line to your figure using:

```
lines(x = x_vec, y = y_vec_optim, col = "orange")
```

Does it fit reasonably well?

Checking our results

Now check the mle2 results by plotting them in another color.
You can modify the code we just used:

```
y_vec_optim <- opt2$par["a"] /  
               (1 + opt2$par["a"] * opt2$par["h"] * x_vec)  
  
lines(x = x_vec, y = y_vec_optim, col = "orange")
```

Hint: mle2 stores parameters using @coef instead of \$par

Your turn!

How well would a linear function with a normal distribution fit these data?

Try to work out how you would write a normal NLL function, and then optimize

$X = \text{Initial \#}$

$Y = \text{predation rate (\# killed / Initial \$)}$

Hint: sd needs to be positive, so you can optimize e^{sd}

`sd = exp(params[3])`

Remainder of Class:

- Work through the example in 6.2.1.2

Before next class:

- 6.4 (6.4.2 optional),
- Skim 6.5
- Read 6.3